Algorithm

1. Start.
2. Read the size of the array `size`.
3. Declare the variable-length array `nums[size]`.
4. Read all positive integer elements into `nums`.
5. Read `target`.
6. Pass the address of `nums` to `two_sum()`.
7. In `two_sum()`:
   - Use an outer loop from `i = 0` to `size - 2`.
   - Use an inner loop from `j = i + 1` to `size - 1`.
   - Calculate the sum of `nums[i]` and `nums[j]` using pointers.
   - If the sum equals `target`, display `i` and `j` and return.
8. If both loops finish without finding a pair, display `-1 -1`.
9. Stop.

Test Case Table

Test	Array	Target	Expected Output	Reason
1	`2 7 11 15`	9	`0 1`	2 + 7 = 9
2	`3 2 4`	6	`1 2`	2 + 4 = 6
3	`3 3`	6	`0 1`	Two distinct indices contain 3
4	`1 2 3`	7	`-1 -1`	No pair has sum 7
5	`5 10 20 30`	35	`0 3`	5 + 30 = 35
6	`1 2`	3	`0 1`	Minimum pair case
7	`4 8 12 16`	50	`-1 -1`	No valid pair

C program

```
#include <stdio.h>


void two_sum(int *nums, int size, int target)

{

    int i, j;
```

```
    for (i = 0; i < size - 1; i++)
    {
        for (j = i + 1; j < size; j++)
        {
            if (*(nums + i) + *(nums + j) == target)
            {
                printf("%d %d\n", i, j);
                return;
            }
        }
    }

    printf("-1 -1\n");
}

int main()
{
    int size, i, target;

    printf("Enter size of array: ");
    scanf("%d", &size);

    int nums[size];
```

```
    printf("Enter array elements: ");
    for (i = 0; i < size; i++)
    {
        scanf("%d", &nums[i]);
    }

    printf("Enter target: ");
    scanf("%d", &target);

    two_sum(nums, size, target);

    return 0;
}
```

Execution

```
[bhavya@Bhavyas-MacBook-Air ~ % nano day61.c
[bhavya@Bhavyas-MacBook-Air ~ % clang day61.c -o day61
[bhavya@Bhavyas-MacBook-Air ~ % ./day61
 Enter size of array: 4
 Enter array elements: 2 7 11 15
 Enter target: 9
 0 1
 bhavya@Bhavyas-MacBook-Air ~ %
```